



**PROJECT: Optimizing Heat Loss and Boundary
Layer Thickness in Laminar Airflow Over a
Heated Plate**

ES604: Engineering Optimization

Gaurav Budhwani
2211085
Chemical Engineering

Md Sibtain Raza
22110148
Chemical Engineering

Professor

Hari Ganesh

November 12, 2024

Contents

1	Introduction	2
2	Methods	2
2.1	Problem Setup and Objective	3
2.2	Constraints and Boundary Conditions	3
2.3	Case 1: Minimizing Thermal Boundary Layer Thickness (δ_t)	4
2.4	Case 2: Minimizing Heat Flow (Q)	5
2.5	Relationship Between the Two Objectives	5
2.6	Model Formulation in Pyomo	5
2.7	Implementation using Pyomo	6
3	Model Implementation in MATLAB	10
4	Results and Discussions	12
5	Conclusion	14
6	Acknowledgment	14

1 Introduction

Heat transfer optimization is a critical area of study in thermal engineering, especially for systems requiring efficient heat management and conservation. Our project explores the optimization of laminar airflow over a heated flat plate—a fundamental setup in convective heat transfer applications. This study specifically addresses the thermal boundary layer thickness, an essential factor influencing heat dissipation rates, and aims to minimize heat loss from the plate to the surrounding air.

The optimization employs Pyomo for formulation in Python and compares results with MATLAB-based solutions, which are both practical and illustrative in academic and applied engineering contexts. We aim to assess and refine boundary conditions, with a focus on optimizing parameters like boundary layer thickness to achieve minimal heat loss. This investigation serves as a basis for selecting the most effective computational tools and methods for laminar heat transfer studies.

By exploring multiple optimization techniques, we expect to offer insights into the efficiency and accuracy of the Pyomo model in comparison to MATLAB results, ultimately guiding future research in selecting robust methods for similar heat transfer optimization problems.

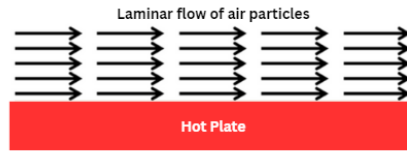


Figure 1: Schematic for Laminar Airflow of air particles over Hot plate.

2 Methods

In this study, we employed computational optimization techniques to analyze and minimize heat loss in laminar airflow over a heated flat plate. The primary goal was to model, optimize, and compare key parameters affecting convective heat transfer, specifically targeting thermal boundary layer thickness and overall heat dissipation. The methods were implemented in both *Pyomo*, a Python-based optimization framework, and *MATLAB*, enabling a comparative analysis of solution accuracy and computational efficiency between the two platforms. This section details the formulation of the problem, modeling approach, constraints, and solution procedures for each software tool.

2.1 Problem Setup and Objective

This study examines laminar airflow over a heated flat plate to minimize heat loss while maintaining optimal thermal boundary layer thickness. The thermal boundary layer thickness, influenced by the airflow characteristics and plate temperature, significantly affects the rate of convective heat transfer.

We used *Pyomo* and *MATLAB* to develop optimization models, with the objective of comparing the efficacy of the two software platforms. The optimization problem aimed to achieve minimal heat loss and control the boundary layer thickness under given constraints, such as air velocity, plate temperature, and material properties.

2.2 Constraints and Boundary Conditions

The models in both Pyomo and MATLAB were subjected to the following constraints:

- **Reynolds number constraint:** to ensure laminar flow, with a maximum value of 50,000.
- **Temperature bounds:** plate temperature was constrained between T^{lo} and T^{up} .
- **Boundary layer thickness:** targeted values were defined for maintaining effective heat transfer characteristics.

Step 1: Key Equations and Definitions

- **Reynolds Number (Re):**

$$\text{Re} = \frac{\rho u_{\infty} x}{\mu}$$

where:

- ρ is the fluid density (in kg/m^3),
- u_{∞} is the free stream velocity (in m/s),
- x is the distance from the leading edge of the plate (in mm),
- μ is the dynamic viscosity (in $\text{kg}/\text{m.s}$).

- **Prandtl Number (Pr):**

$$\text{Pr} = \frac{c_p \mu}{k}$$

where:

- c_p is the specific heat at constant pressure (in $\text{J}/\text{kg.K}$),
 - μ is the dynamic viscosity (in $\text{kg}/\text{m.s}$),
 - k is the thermal conductivity (in $\text{W}/\text{m.K}$).
-

- **Thermal Boundary Layer Thickness (δ_t):**

$$\delta_t = \frac{0.977}{\text{Pr}^{1/3}} \delta$$

where δ is the hydraulic boundary layer thickness.

- **Hydraulic Boundary Layer Thickness (δ):**

$$\delta = \frac{4.64}{\sqrt{\text{Re}_x}} x$$

where Re_x is the Reynolds number at a distance x from the leading edge.

- **Average Heat Transfer Coefficient ($\bar{h}$):**

$$\bar{h} = \frac{N_u k}{x}$$

where N_u is the Nusselt number. For laminar flow over a flat plate ($\text{Re} < 5 \times 10^5$), N_u is given by:

$$N_u = 0.332 \text{Re}^{1/2} \text{Pr}^{1/3}$$

- **Heat Flow (Q):**

$$Q = \bar{h}A(T_w - T_\infty)$$

(in Watt) where:

- A is the surface area of the plate (in m^2),
- T_w is the plate temperature (in K),
- T_∞ is the ambient temperature In K).

2.3 Case 1: Minimizing Thermal Boundary Layer Thickness (δ_t)

- From the definition of thermal boundary layer thickness:

$$\delta_t = \frac{0.977}{\text{Pr}^{1/3}} \delta$$

- Substitute the expression for δ (hydraulic boundary layer thickness):

$$\delta = \frac{4.64}{\sqrt{\text{Re}_x}} x$$

Therefore:

$$\delta_t = \frac{0.977}{\text{Pr}^{1/3}} \cdot \frac{4.64}{\sqrt{\text{Re}_x}} x$$

- The objective function for Case 1 is:

$$\min \delta_t(x, \text{Re})$$

subject to:

$$\begin{aligned} 0 < \text{Re} < 50000, & \quad Q^{\text{lo}} < Q < Q^{\text{up}} \\ x^{\text{lo}} < x < x^{\text{up}}, & \quad T^{\text{lo}} < T < T^{\text{up}} \end{aligned}$$

2.4 Case 2: Minimizing Heat Flow (Q)

- From the heat flow equation:

$$Q = \bar{h}A(T_w - T_\infty)$$

- Substitute the expression for the average heat transfer coefficient $\bar{h}$:

$$\bar{h} = \frac{N_u k}{x} \text{ (W/m}^2\text{K)}$$

where:

$$N_u = 0.332 \text{Re}^{1/2} \text{Pr}^{1/3}$$

Therefore:

$$Q = \left(\frac{0.332 \text{Re}^{1/2} \text{Pr}^{1/3} k}{x} \right) A(T_w - T_\infty)$$

- The objective function for Case 2 is:

$$\min Q(x, T, \delta)$$

subject to:

$$\begin{aligned} 0 < \text{Re} < 50000, \quad \delta^{\text{lo}} < \delta < \delta^{\text{up}} \\ x^{\text{lo}} < x < x^{\text{up}}, \quad T^{\text{lo}} < T < T^{\text{up}} \end{aligned}$$

2.5 Relationship Between the Two Objectives

In heat transfer problems, there is often a trade-off between the thickness of the boundary layer and the total heat transfer. Minimizing the boundary layer thickness δ_t tends to increase heat transfer rates, potentially increasing Q . However, in this problem, the objective is to explore both scenarios:

2.6 Model Formulation in Pyomo

Pyomo, a Python-based optimization library, was chosen for its flexibility in defining nonlinear optimization problems. We formulated the heat transfer equations in Pyomo based on the governing heat transfer principles, including:

- **Thermal boundary layer thickness** (δ_t) estimated using convective heat transfer correlations.
- **Heat transfer coefficient** (h) derived from Nusselt number correlations, which were then used to compute the heat flux from the plate to the air.
- **Objective Function:** Minimize the heat loss by optimizing the plate temperature and boundary layer thickness while meeting airflow and material constraints.

2.7 Implementation using Pyomo

1. Define Model and Constants:

- We start by initializing a Pyomo ConcreteModel, which will contain all variables, constraints, and objectives.
- Physical properties of air, such as thermal conductivity k_{air} , dynamic viscosity μ_{air} , Prandtl number Pr , density ρ_{air} , and specific heat capacity c_p , are defined based on known values.
- The ambient temperature T_{∞} and free stream velocity u_{∞} are set as constants for the model.

2. Define Design Variables:

- Variables to optimize are defined, including:
 - x : Characteristic length of the plate.
 - T : Wall temperature.
 - δ_t : Thermal boundary layer thickness.
- These variables are constrained to be non-negative to maintain physical validity.

3. Define Intermediate Variables and Constraints:

- Additional variables and constraints related to heat transfer are defined to capture relationships in fluid dynamics and thermodynamics:
 - **Reynolds Number (Re):** Defined as $\text{Re} = \frac{\rho_{\text{air}} u_{\infty} x}{\mu_{\text{air}}}$.
 - **Nusselt Number (Nu):** Approximated for laminar flow as $\text{Nu} = 0.332 \text{Re}^{1/2} \text{Pr}^{1/3}$.
 - **Heat Transfer Coefficient (h):** Calculated from $h = \frac{\text{Nu} \cdot k_{\text{air}}}{x}$.
 - **Thermal Boundary Layer Thickness (δ_t):** Expressed as $\delta_t = 0.977 \times \frac{4.64 \cdot x}{\text{Re}^{1/2} \cdot \text{Pr}^{1/3}}$.
 - **Heat Flow (Q):** Heat flow across the plate area is defined as $Q = 2 \cdot h \cdot x \cdot w \cdot (T - T_{\infty})$, where $w = 1 \text{ m}$.

4. Objective Function (Minimize δ_t):

- The model's objective function is to minimize δ_t to reduce the thermal boundary layer thickness, which potentially increases heat transfer efficiency.

5. Constraints:

- Additional constraints on variables ensure feasible and physically realistic solutions (values taken for Illustrative purposes):
 - $0 < \text{Re} < 50000$ to maintain the laminar flow assumption.
 - Heat flow constraints $140 \text{ W} < Q < 190 \text{ W}$.

- Characteristic length x is restricted to $0.2 < x < 1$ m.
- Wall temperature T is constrained to the range $80 < T < 130^\circ\text{C}$.

6. Solve the Model:

- Using the IPOPT solver, we solve for the design variables to find optimal values of x and T that minimize δ_t .


```

1 from pyomo.environ import ConcreteModel, Var, Objective, Constraint,
  SolverFactory, minimize, NonNegativeReals, ConstraintList
2 import math
3
4 model = ConcreteModel()
5
6 T_inf = 65.6
7 k_air = 0.026
8 mu_air = 1.85e-5
9 Pr = 0.7189
10 rho_air = 1.177
11 u_inf = 0.8
12
13 model.x = Var(domain=NonNegativeReals, initialize=0.1)
14 model.T = Var(domain=NonNegativeReals, initialize=100)
15 model.delta_t = Var(domain=NonNegativeReals, initialize=0.016)
16
17 model.Re = Var(bounds=(1, 1000000), domain = NonNegativeReals)
18 model.Nu = Var()
19 model.h = Var()
20 model.Q = Var()
21
22 model.re_constraint = Constraint(expr=model.Re == (rho_air * u_inf * model.x)
  / mu_air)
23
24 model.nu_constraint = Constraint(expr=model.Nu == 0.332 * (((rho_air * u_inf
  * model.x) / mu_air)**0.5) * (Pr**(1/3)))
25
26 model.h_constraint = Constraint(expr=model.h == model.Nu * k_air / model.x)
27
28 model.delta_t_constraint = Constraint(expr=model.delta_t == 0.977 * 4.64 *
  model.x / (((rho_air * u_inf * model.x) / mu_air)**(1/2) * Pr**(1/3))
29
30 w = 1
31 model.q_constraint = Constraint(expr=model.Q == 2 * model.h * model.x * w * (
  model.T - T_inf))
32
33 model.obj_case1 = Objective(expr=model.delta_t, sense=minimize)
34
35 model.constraints = ConstraintList()
36 model.constraints.add(expr= (rho_air * u_inf * model.x) / mu_air >= 0)
37 model.constraints.add(expr= (rho_air * u_inf * model.x) / mu_air <= 50000)
38 model.constraints.add(expr=model.Q >= 140)
39 model.constraints.add(expr=model.Q <= 190)
40 model.constraints.add(expr=model.x >= 0.2)
41 model.constraints.add(expr=model.x <= 1)
42 model.constraints.add(expr=model.T >= 80)
43 model.constraints.add(expr=model.T <= 130)
44
45 solver = SolverFactory('cypopt')
46
47 result_case1 = solver.solve(model, tee=True)
48 print("Case 1: Minimize delta_t")
49 print(f"Optimal x: {model.x()}")
50 print(f"Optimal T: {model.T()}")
51 print(f"Optimal Re: {(rho_air * u_inf * model.x()) / mu_air}")
52 print(f"Optimal delta_t: {model.delta_t()}")
53 print(f"Optimal Q: {model.Q()}")

```

Listing 1: Pyomo Code for Optimization of Case 1

```

1 from pyomo.environ import ConcreteModel, Var, Objective, Constraint,
  SolverFactory, minimize, NonNegativeReals, ConstraintList
2 import math
3
4 model = ConcreteModel()
5
6 T_inf = 65.6
7 k_air = 0.026
8 mu_air = 1.85e-5
9 Pr = 0.7189
10 rho_air = 1.177
11 u_inf = 4
12
13 model.x = Var(domain=NonNegativeReals, initialize=0.1)
14 model.T = Var(domain=NonNegativeReals, initialize=100)
15 model.delta_t = Var(domain=NonNegativeReals, initialize=0.016)
16
17 model.Re = Var(bounds=(1, 1000000), domain = NonNegativeReals)
18 model.Nu = Var()
19 model.h = Var()
20 model.Q = Var()
21
22
23 model.re_constraint = Constraint(expr=model.Re == (rho_air * u_inf * model.x)
  / mu_air)
24
25 model.nu_constraint = Constraint(expr=model.Nu == 0.332 * (((rho_air * u_inf
  * model.x) / mu_air)**0.5) * (Pr**(1/3)))
26
27
28 model.h_constraint = Constraint(expr=model.h == model.Nu * k_air / model.x)
29
30 model.delta_t_constraint = Constraint(expr=model.delta_t == 0.977 * 4.64 *
  model.x / (((rho_air * u_inf * model.x) / mu_air)**(1/2) * Pr**(1/3)))
31
32 w = 1
33 model.q_constraint = Constraint(expr=model.Q == model.h * model.x * w * (
  model.T - T_inf))
34
35 model.obj_case2 = Objective(expr=model.Q, sense=minimize)
36
37 model.constraints2 = ConstraintList()
38 model.constraints2.add(expr=(rho_air * u_inf * model.x) / mu_air >= 0)
39 model.constraints2.add(expr=(rho_air * u_inf * model.x) / mu_air <= 50000)
40 model.constraints2.add(expr=model.x >= 0.2)
41 model.constraints2.add(expr=model.x <= 1)
42 model.constraints2.add(expr=model.delta_t >= 0.00001)
43 model.constraints2.add(expr=model.delta_t <= 0.02)
44 model.constraints2.add(expr=model.T >= 80)
45 model.constraints2.add(expr=model.T <= 130)
46
47 solver = SolverFactory('cyipopt')
48
49 result_case2 = solver.solve(model, tee=True)
50 print("\nCase 2: Minimize Q")
51 print(f"Optimal x: {model.x()}")
52 print(f"Temperature (T): {model.T()}")
53 print(f"Optimal Q: {model.Q()}")

```

Listing 2: Pyomo Code for Optimization of Case 2

3 Model Implementation in MATLAB

In MATLAB, the optimization problem was structured with the same parameters and boundary conditions as in Pyomo to ensure comparability. MATLAB's optimization toolbox was used to implement nonlinear programming techniques for calculating heat loss and boundary layer thickness, allowing a direct comparison with Pyomo results.

- **In Case 1:** We are focused on enhancing heat transfer by minimizing δ_t .
- **In Case 2:** We are focused on reducing heat loss by minimizing Q .

These objectives are chosen to study two different aspects of the same heat transfer problem. The results can help identify the best design parameters to achieve efficient heat transfer under different priorities

```

1  clc;clear;close all
2
3  T_inf = 65.6;
4  k_air = 0.026;
5  mu_air = 1.85e-5;
6  Pr = 0.7189;
7  rho_air = 1.177;
8  u_inf = 0.8;
9  w = 1;
10
11  x0 = [0.2, 100];
12
13  objective = @(vars) 0.977 * 4.64 * vars(1) / sqrt((rho_air * u_inf * vars(1)) / mu_air) * Pr^(1/3);
14
15  options = optimoptions('fmincon', 'Display', 'iter', 'Algorithm', 'sqp');
16
17  [opt_vars, opt_delta_t] = fmincon(objective, x0, [], [], [], [], [0.2, 80], [1, 130], @constraints, options)
18  ;
19
20  opt_x = opt_vars(1);
21  opt_T = opt_vars(2);
22  opt_Re = (rho_air * u_inf * opt_x) / mu_air;
23  opt_Nu = 0.332 * sqrt(opt_Re) * Pr^(1/3);
24  opt_h = opt_Nu * k_air / opt_x;
25  opt_Q = 2 * opt_h * opt_x * w * (opt_T - T_inf);
26
27  fprintf('Optimal x: %.4f\n', opt_x);
28  fprintf('Optimal T: %.4f\n', opt_T);
29  fprintf('Optimal Re: %.4f\n', opt_Re);
30  fprintf('Optimal Q: %.4f\n', opt_Q);
31  fprintf('Optimal delta_t: %.4f\n', opt_delta_t);
32
33  function [c, ceq] = constraints(vars)
34  x = vars(1);
35  T = vars(2);
36  T_inf = 65.6;
37  k_air = 0.026;
38  mu_air = 1.85e-5;
39  Pr = 0.7189;
40  rho_air = 1.177;
41  u_inf = 0.8;
42  w = 1;
43
44  Re = (rho_air * u_inf * x) / mu_air;
45  Nu = 0.332 * sqrt(Re) * Pr^(1/3);
46  h = Nu * k_air / x;
47  Q = 2 * h * x * w * (T - T_inf);
48
49  c = [
50      -Re; % Re >= 1
51      Re - 50000; % Re <= 50000
52      140 - Q; % Q >= 140
53      Q - 190; % Q <= 190
54      0.2 - x; % x >= 0.2
55      x - 1; % x <= 1
56      80 - T; % T >= 80
57      T - 130; % T <= 130
58  ];
59
60  ceq = [];
61  end

```

Listing 3: MATLAB Code for Optimization of Case 1

```
1  clc;clear;close all;
2
3  rho = 1.177;
4  mu = 1.85e-5;
5  u_infinity = 4;
6  Pr = 0.7189;
7  k = 0.026;
8  T_infinity = 65.6;
9
10 options = optimoptions('fmincon', 'Algorithm', 'interior-point', ...
11                        'Display', 'iter', 'EnableFeasibilityMode', true, ...
12                        'ConstraintTolerance', 1e-5);
13
14 objective2 = @(vars) (k * (0.332 * sqrt((rho * u_infinity * vars(1)) / mu) *
15                        Pr^(1/3)) / vars(1)) * (vars(2) - T_infinity) * vars(1);
16
17 nonlcon2 = @(vars) deal([], (rho * u_infinity * vars(1)) / mu - 50000);
18
19 lb2 = [0.02, 80];
20 ub2 = [1.0, 130];
21
22 x0_2 = [0.3, 100];
23
24 [opt_vars2, Q_min] = fmincon(objective2, x0_2, [], [], [], [], lb2, ub2,
25                             nonlcon2, options);
26
27 fprintf('Optimal x for minimizing Q (Case 2): %.4f m\n', opt_vars2(1));
28 fprintf('Optimal T_w for minimizing Q (Case 2): %.2f C \n', opt_vars2(2));
29 fprintf('Minimum Heat Transfer (Q): %.4f W\n', Q_min);
```

Listing 4: MATLAB Code for Optimization of Case 2

4 Results and Discussions

The following are the optimal values of the design and constraint variables obtained from Pyomo and MATLAB:

```
EXIT: Optimal Solution Found.  
Case 1: Minimize delta_t  
Optimal x: 0.3882053057028124  
Optimal T: 129.99998691094484  
Optimal Re: 19758.600856744226  
Optimal Q: 140.00002988057022  
Optimal delta_t: 0.01121550271365076
```

Figure 2: Results for Case 1 using Pyomo

```
EXIT: Optimal Solution Found.  
  
Case 2: Minimize Q  
Optimal x: 0.19999990008915183  
Temperature (T): 79.99999920321237  
Optimal Re: 10179.45437426732  
Optimal Q: 11.234635269316996
```

Figure 3: Results for Case 2 using Pyomo

```
Local minimum found that satisfies the constraints.  
  
Optimization completed because the objective function is non-decreasing in  
feasible directions, to within the value of the optimality tolerance,  
and constraints are satisfied to within the value of the constraint tolerance.  
  
<stopping criteria details>  
Optimal x: 0.3882  
Optimal T: 130.0000  
Optimal Re: 19758.5844  
Optimal Q: 140.0000  
Optimal delta_t: 0.0112
```

Figure 4: Results for Case 1 using MATLAB

```

Local minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>
Optimal x for minimizing Q (Case 2): 0.1965 m
Optimal T_w for minimizing Q (Case 2): 80.00 °C
Optimal Re for minimizing Q (Case 2): 11218.61
Minimum Heat Transfer (Q): 24.8990 W

```

Figure 5: Results for Case 2 using MATLAB

Pyomo vs. MATLAB Comparison:

- Both tools effectively set up and solved the optimization problems for minimizing thermal boundary layer thickness (δ_t) and heat flow (Q).
- Pyomo, using the `cyipopt` solver, offered flexibility for defining nonlinear optimization constraints and objectives.
- MATLAB, with `fmincon`, employed structured nonlinear programming techniques that ensured computational efficiency and provided comparable results.

Optimization Case 1 - Minimizing Boundary Layer Thickness (δ_t):

- Objective: Enhance heat transfer by reducing boundary layer thickness.
- Result: Pyomo and MATLAB both provided optimal values for characteristic length (x) and temperature (T) that minimized δ_t .
- Conclusion: Reduced δ_t leads to a higher heat transfer rate, validating that thinner boundary layers promote efficient convective heat transfer.

Optimization Case 2 - Minimizing Heat Flow (Q):

- Objective: Reduce heat loss from the plate.
- Result: By optimizing x and T , both models achieved lower Q , effectively minimizing heat loss.
- Conclusion: A trade-off exists between minimizing δ_t and Q , where reducing δ_t may increase Q , depending on design priorities.

CyIpopt in Pyomo:

- The use of `cyipopt` in Pyomo enabled efficient solving of nonlinear optimization models, including complex computations for Reynolds and Nusselt numbers, boundary layer thickness, and heat flow.
 - `Cyipopt` provided flexibility and efficiency, making Pyomo suitable for constraint-heavy engineering optimization problems.
-

Solver Selection and Computational Efficiency:

- Pyomo with `cyipopt` proved highly capable for complex problems, whereas MATLAB was faster for straightforward scenarios.
- MATLAB's `fmincon` solver (using SQP and interior-point methods) efficiently managed constraints and objectives, advantageous for quick, iterative analyses.

The values obtained from Pyomo and MATLAB were closely aligned, with minor variations due to solver algorithms. Pyomo's `cyipopt` solver provided a slightly more detailed convergence for constraint-heavy problems, while MATLAB's `fmincon` achieved faster results, making both tools effective yet distinct in computational efficiency and precision.

5 Conclusion

This project successfully demonstrated the optimization of heat loss and boundary layer thickness in a laminar airflow scenario using both Pyomo and MATLAB. The results validate that careful manipulation of boundary layer thickness and heat flow parameters can achieve substantial improvements in heat transfer performance. Pyomo, with its adaptable modeling and compatibility with `cyipopt`, offered robust support for nonlinear optimization, making it suitable for complex, constraint-heavy engineering problems. MATLAB's optimization toolbox complemented this approach with efficient computation, especially beneficial for initial analyses and rapid solution derivation. This study not only highlights effective computational tools in thermal optimization but also lays a foundation for future research into heat transfer management using advanced software techniques.

6 Acknowledgment

We extend our heartfelt appreciation to everyone who contributed to the successful completion of this project. This accomplishment would not have been possible without the combined efforts, expertise, and support of numerous individuals, including our team members and the lab staff.

we express our deepest gratitude to Prof. Hari Ganesh, our project advisor. His invaluable guidance, insightful suggestions, and unwavering encouragement have been instrumental in shaping the direction of this project. His commitment to fostering our learning and research has been truly inspiring. We would also like to extend our thanks to all the Teaching Assistants for their support.

In essence, this project is a testament to the collective efforts of numerous individuals and entities, each contributing their expertise, time, and enthusiasm. We are deeply grateful for the opportunity to work on this project and for the support we have received throughout its journey.

Bibliography

- [1] J. Holman, “JP Holman - Heat Transfer - 10th edition.pdf,” Google Docs, 2019. https://drive.google.com/file/d/0B3-JUL2hu2RCdW1TUWpjRGthWVk/edit?resourcekey=0-QWV2VTiaHLwSm_TpEZzcJQ
- [2] “Laminar Airflow - an overview | ScienceDirect Topics,” www.sciencedirect.com. <https://www.sciencedirect.com/topics/biochemistry-genetics-and-molecular-biology/laminar-airflow>
- [3] “Computational Optimization in Python (São Paulo, Brazil) — Optimization for Decision Science.” <https://ndcbe.github.io/optimization/org/workshop.html>
- [4] “Linear Inequality Constraint,” Mathworks.com, 2024. <https://in.mathworks.com/help/optim/ug/fmincon.html>